

Updater

Dieses Arbeitsblatt wird etwas geleitet gestaltet inkl. einiger kleiner Übungen

Zunächst zeichnen wir eine Zahlenebene und eine abschnittsweise definierte Funktion ein. Dabei betrachten wir die Funktion f mit

$$f(x) = \begin{cases} 1/2 x & \text{für } x \leq 2 \\ -x + 2 & \text{für } x > 2. \end{cases}$$

```
ax = Axes(x_range=[-5, 5], y_range=[-3, 3])
graph1 = ax.plot(lambda x: 0.5*x, color=BLUE, x_range=[-5, 2])
graph2 = ax.plot(lambda x: -1*x+2, color=BLUE, x_range=[2, 5])

self.play(Create(ax))
self.play(Create(graph1))
self.play(Create(graph2))
```

Nun wollen wir einen Punkt im Ursprung einzeichnen lassen.

```
dot = Dot(ax.c2p(0,0))
```

Mit `ax.c2p(x,y)` können wir zu x und y Koordinaten eines Punktes die Positionen auf der Manim Ebene erhalten.

Es wäre doch schön, wenn wir den Punkt über die Sprungstelle laufen lassen könnten, um zu visualisieren, dass da tatsächlich ein Sprung passiert. *Wir wackeln etwas an x und damit wackelt es etwas sehr an y .*

Aufgabe 1

Wir könnten überlegen, ob wir `MoveAlongPath` verwenden. Probiert das doch kurz selbst aus.

Wir sehen aber, dass wir das „Hin- und Herwackeln“ nicht so schön umsetzen können. Wir brauchen also eine andere Lösung.

Zunächst erstellen wir eine Funktion, die unserer o.g. Funktion f entspricht:

```
def func(x):
    if int(x) < 2:
        return 0.5*x
    else:
        return -1*x+2
```

Nun kommen wir zu einem Konzept, was wir bisher noch nicht besprochen haben. Dabei handelt es sich um Updater. Das sind Funktionen, die wir einem Mobject „anheften“ können und die jeden Frame ausgeführt wird. Wir wollen den Punkt `dot` zu jedem Frame auf den Punkt $(x, func(x))$ bewegen.

```
def update_function(mobj):  
    mobj.move_to(ax.c2p(...))
```

Aber welche Werte tragen wir dort ein? Wir haben in Manim ValueTracker, die dafür ausgelegt sind, einen Wert zu speichern, den man im Nachhinein dynamisch ändern kann. ValueTracker haben den Vorteil, dass sie nicht wie Variablen den Wert direkt beim setzen bspw. von 0 auf 1 ändern, sondern dynamisch über eine `run_time` hinweg.

Das ermöglicht es uns, die x -Komponente zu variieren.

```
t = ValueTracker(0)  
  
def update_function(mobj):  
    mobj.move_to(ax.c2p(t.get_value(), func(t.get_value())))
```

Und damit sind wir fast fertig. Nun müssen wir unserem `dot` noch sagen, dass er nun einen Updater hat, den er regelmäßig ausführen soll. Das machen wir mit

```
dot.add_updater(update_function)
```

Und nun können wir mit `self.play(t.animate.set_value(4))` den Wert von `t` auf 4 setzen. Mit `run_time=5` können wir die Animation noch verlangsamen. Nun lassen wir den Punkt um die Sprungstelle wandern:

```
xs = [1.5, 2.4, 1.8, 2.1, 1.95, 2.01, 1.99]  
for x in xs:  
    self.play(t.animate.set_value(x), run_time=1)
```

Mit `dot.remove_updater(updater)` kann man einzelne Updater und mit `dot.clear_updater()` kann man alle Updater entfernen.

Aufgaben:

Aufgabe 2

Betrachte folgenden Code. Was macht der rote Punkt?

```
class UpdaterBasic(Scene):
    def construct(self):
        dot = Dot(color=RED)

        def move_right(mobj, dt):
            mobj.shift(RIGHT * dt)

        dot.add_updater(move_right)
        self.add(dot)
        self.wait(3)
```

In welche Richtung bewegt sich der Punkt? Was passiert, wenn man `dt` durch `2*dt` ersetzt?

Aufgabe 3

Ein Kreis bewegt sich auf dem Bildschirm. Ein Text soll immer über dem Kreis stehen, egal wohin er wandert. Ergänze den fehlenden Updater.

```
class LabelFollowsCircle(Scene):
    def construct(self):
        circle = Circle().shift(LEFT)
        label = Text("Ich folge").next_to(circle, UP)
        self.add(circle, label)

        # TODO: Ergänze hier einen Updater, der label an circle bindet

        # Kreis bewegt sich nach rechts
        self.play(circle.animate.shift(4 * RIGHT), run_time=4)
```

Aufgabe 4

Erstelle ein Quadrat, das sich auf einer horizontalen Linie bewegt. Wenn es am rechten Rand ankommt, soll es zurückspringen. Nutze Updater und einfache Bedingungen.



This document is subject to the Creative Commons Zero (CC0) License.
To create this document, we used $\text{\LaTeX}$.